W3.io Protocol

Verifiable Workflow Execution with On-Chain Settlement

Audie Sheridan Ben Murray David Post

2026

Contents

1 Abstract

W3.io is a workflow execution protocol that connects fragmented Web3 infrastructure into verifiable, multi-step workflows that power real-world applications. Developers compose workflows from modular actions — blockchain operations across Ethereum, Solana, and Bitcoin, data queries, AI inference, payments, compliance checks, and dozens of other ecosystem partner capabilities — using a declarative YAML syntax compatible with GitHub Actions. The protocol transforms isolated infrastructure components into programmable, composable pipelines where each step's execution is independently verifiable.

A distributed network of validators executes each workflow step, reaches Byzantine fault-tolerant consensus on the result, and produces cryptographic settlement receipts. These receipts are accumulated into epoch commitments anchored to an EVM L1, enabling any party to verify that a specific workflow executed correctly without re-executing it or trusting the executor.

The W3 token gates participation in the network. Ecosystem partners post collateral to operate subnets, solution builders stake to deploy applications, sales teams stake to earn rewards for driving

adoption and end-user conversions, and validators bond tokens to secure consensus. Revenue from workflow execution flows in stablecoins, separating the utility token from the payment medium. Rewards are tied to active contribution — execution, verification, building, and adoption — not passive holding. The token does not represent equity, debt, or a claim on protocol revenues.

This paper describes the architecture, workflow execution model, consensus mechanism, settlement layer, namespace model, validator infrastructure, token utility, and security properties of the W3.io protocol. The protocol is implemented in Rust, open-source, and deployed on testnet with over 200,000 workflow executions per day.

2 Introduction

2.1 The Integration Gap

Decentralized infrastructure has matured rapidly. Individual services now exist for payments, data, compute, storage, identity, oracles, and security. Multiple blockchains provide settlement for different use cases (Nakamoto 2008; Buterin 2014; Yakovenko 2018). The building blocks are there.

What doesn't exist is a way to connect these services into verifiable end-to-end workflows. A business process that requires a payment, a compliance check, a data query, and an on-chain settlement today requires custom integration code for each service, separate authentication and billing relationships with each provider, and no unified way to verify that the entire process executed correctly.

This gap has three dimensions.

Centralization by default. When developers need to orchestrate multiple services, the path of least resistance is a centralized server calling each API sequentially. This contradicts the decentralized properties of the underlying services, introduces single points of failure, and creates vendor lock-in with cloud providers. The result is decentralized infrastructure coordinated by centralized glue code.

Integration complexity. Each decentralized service has its own API patterns, authentication mechanisms, data formats, and error handling conventions. Integrating even two services requires specialized knowledge of both. Integrating five or ten, as real business processes demand, multiplies the development cost, the testing surface, and the ongoing maintenance burden. Most Web3 development is bespoke, built from scratch against the same patterns that every other team is also building from scratch.

Business friction. Beyond the technical challenges, using multiple decentralized providers means separate legal agreements, separate payment arrangements, and separate vendor relationships. Discovery is also a problem. Hundreds of decentralized projects exist, but their capabilities and

integration surfaces are poorly documented and difficult to evaluate without deep technical engagement.

These challenges are not hypothetical. They are the reason most production Web3 applications still run their off-chain logic on AWS, even when the on-chain components are fully decentralized.

2.2 W3.io's Approach

W3.io addresses the integration gap with a protocol-level execution layer that connects decentralized services into verifiable workflows. Rather than requiring developers to write custom integration code, W3.io provides a declarative workflow language where each step invokes a pre-integrated action from the W3.io ecosystem.

W3.io's model has three layers.

Actions are modular capabilities provided by ecosystem partners. An action encapsulates a specific operation behind a standard interface: querying a database, executing a blockchain transaction, running an AI model, sending a payment. Actions are versioned, namespace-scoped, and independently deployable. W3.io ships with native actions for Ethereum, Solana, and Bitcoin blockchain operations. Partners contribute actions for their specific capabilities.

Workflows are sequences of actions triggered by events. A workflow definition specifies a trigger (a cron schedule, an RPC call, a blockchain event, or an oracle feed), a series of steps that each invoke an action, and the data flow between steps. Workflows are compiled from YAML into a canonical form, deployed to the validator network, and executed with Byzantine fault-tolerant consensus on each step's result.

Solutions are workflows integrated into end-user applications. A treasury management application, a compliance pipeline, a cross-chain bridge: each is a solution built from workflows that compose ecosystem actions. Solutions inherit the verifiability of the underlying protocol without requiring the end user to understand the execution mechanics.

This layered model means that ecosystem partners contribute capabilities once, developers compose them into workflows without custom integration code, and businesses deploy solutions that are verifiable by construction.

2.3 What This Paper Covers

The remainder of this paper describes the W3.io protocol in detail:

- **Architecture** (Section 2): system overview, data flow, and the relationship between protocol components
- **Workflows** (Section 3): the declarative syntax, trigger types, step kinds, and expression

evaluation

- **Execution** (Section 4): how steps are isolated and executed on validators, with timeout enforcement and signal escalation
- **Consensus** (Section 5): the BOSCO BFT protocol, committee selection, and the four contexts in which consensus is used
- **Settlement** (Section 6): the cryptographic commitment structure that makes workflow execution independently verifiable
- **Cryptography** (Section 7): the specific algorithms used and their provenance in production-grade, audited implementations
- **Namespaces** (Section 8): the hierarchical multi-tenancy model for organizing workflows, billing, and access control
- **Validators** (Section 9): how operators join the network, declare capabilities, and maintain liveness
- **Ecosystem** (Section 10): the B2B2C partner model and the action registry
- **Token** (Sections 11-12): the utility of the W3 token within the protocol and its economic structure
- **Governance** (Section 13): how the protocol evolves and the decentralization roadmap
- **Security** (Section 14): the threat model, BFT assumptions, and known limitations

W3.io's protocol is implemented in Rust (Matsakis and Klock 2014), open-source, and deployed on testnet. This paper describes the system as designed and implemented, not as aspirational. Where features are planned but not yet deployed, this is stated explicitly.

3 Architecture

3.1 Ingredients, Recipes, and Solutions

W3.io's ecosystem is organized around three tiers of composability.

Ingredients are capabilities provided by individual ecosystem partners. Each partner exposes their capabilities as modular, reusable components within the W3.io ecosystem. Space and Time provides data queries. Circle and Stripe handle payments. Hyperbolic offers AI inference. Chainalysis handles compliance screening. Pyth delivers price feeds. An ingredient is a self-contained unit of functionality that can be invoked as part of a larger process.

Recipes combine two or more ingredients into adaptive logic that addresses a specific business need. A recipe defines the trigger conditions, the sequence of operations, the data flow between steps, and the failure handling behavior. Recipes are W3.io's unit of deployment. A developer writes a recipe, deploys it to the validator network, and it executes automatically when triggered.

Solutions are recipes integrated into end-user applications. A payments product, a compliance pipeline, a yield vault: each is a solution that composes ingredients through recipes to deliver a complete user experience. Solutions inherit the verifiability of the underlying protocol. Every step of every recipe execution is attested by consensus and provable against an on-chain commitment.

This model enables a network effect. Each new ingredient makes every existing recipe potentially more capable, and each new recipe demonstrates a pattern that other builders can adapt.

3.2 Actions and Workflows

At the protocol level, ingredients become **actions** and recipes become **workflows**.

An action has a namespace-scoped identifier, a version, and defined inputs and outputs. W3.io ships with native actions for three blockchain families:

- **Ethereum**: balance queries, token transfers and approvals, contract deployment and calls, event monitoring, NFT operations, ENS name resolution
- **Solana**: balance queries, token accounts, program invocation, transfers, transaction monitoring
- **Bitcoin**: balance queries, UTXO management, fee estimation, transaction construction and broadcast

Ecosystem partners extend the action set by publishing their own actions. These are packaged as container images with a standard entry point, versioned independently, and invoked via the `uses:` step kind in workflow YAML. Current ecosystem actions include data queries (Space and Time), price feeds (Pyth), compliance screening (Chainalysis), payments (Circle, Stripe), AI inference (Hyperbolic), caching (Redis), email delivery, and token transfers.

Workflows are defined in YAML using a syntax compatible with GitHub Actions:

```yaml
name: Cross-chain settlement
on:
  schedule:
    - cron: '*/5 * * * *'
jobs:
  settle:
    steps:
      - name: Query pending transactions
        uses: w3-io/w3-sxt-action@v1
        with:
          command: query
```

```
        sql: >
          SELECT * FROM transactions
          WHERE status = 'pending'

    - name: Screen for compliance
      uses: w3-io/w3-chainalysis-action@v1
      with:
        address: ${{ steps.query.outputs.sender }}

    - name: Execute settlement
      ethereum:
        action: call-contract
        network: ethereum
        params:
          to: "0x..."
          function: "settle(address,uint256)"
          args:
            - ${{ steps.query.outputs.sender }}
            - ${{ steps.query.outputs.amount }}
```

The GitHub Actions compatibility is deliberate. Developers already familiar with CI/CD workflows can write W3.io workflows without learning a new language. The YAML is compiled into a canonical form, hashed for on-chain provenance, and deployed to the validator network.

3.3 The Three-Layer Data Architecture

W3.io separates data responsibilities across three layers, each optimized for its role.

Layer 1: L1 Settlement. The on-chain layer stores commitments, not data. The footprint is minimal: one cumulative root per epoch, plus contract state for the validator set and workflow registry. This keeps gas costs low while providing the anchor for cryptographic verification.

Layer 2: Protocol Nodes. Validators store everything. Full workflow execution traces, receipt storage, epoch accumulation, P2P consensus messages, and a write-ahead log for crash recovery. This layer is the evidence behind the Layer 1 commitments and serves as the data availability layer for proof generation.

Layer 3: Index and Query. An indexer watches Layer 1 events and reconstructs queryable views: workflow runs by namespace, by epoch, by action type. The CLI provides developer access to settlement status and proof export. The API layer supports MCP (Model Context Protocol) for

AI agent integration.

3.4 The Spine and Ribs

W3.io's settlement architecture is best understood as a spine with ribs.

```
The Spine: L1 Epoch Chain

  Epoch 0        Epoch 1        Epoch 2
  root: 0xaaa    root: 0xbbb    root: 0xccc
  prev: 0x000    prev: 0xaaa    prev: 0xbbb

The Ribs: Per-Run Hashchains

  Workflow A -- block 1 -- block 2 -- receipt
  Workflow B -- block 1 -- receipt
  Workflow C -- block 1 -- block 2 -- block 3 -- receipt
```

The **spine** is the L1 epoch chain. A sequential, immutable record of cumulative sparse merkle tree roots (Gao et al. 2021) on-chain. Each epoch's root covers all workflow receipts across all namespaces ever settled, not just the current epoch. The chain of roots captures the full evolution of world state.

The **ribs** are individual workflow execution traces. Per-run hashchains of consensus blocks that branch off the spine. Each step in a workflow produces a block containing the step's inputs, outputs, executor identity, and the previous block's hash. When the workflow completes, the final block hash and metadata are compressed into a receipt. That receipt is inserted into the namespace's sparse merkle tree, which feeds into the global tree, which produces the epoch root committed to the spine.

Any workflow execution is provable against any epoch root that includes it. A verifier needs only the on-chain root, a receipt, and two merkle proofs (receipt-in-namespace, namespace-in-global) to confirm that a specific workflow executed with a specific result at a specific time. No re-execution required. No trust in the validator that ran it.

4 Workflows

4.1 Declarative Syntax

W3.io workflows are defined in YAML using the same syntax as GitHub Actions. This is not a loose compatibility. The W3.io compiler parses the same grammar, supports the same expression lan-

guage, and handles the same structural elements: jobs, steps, conditionals, environment variables, secrets, outputs, and step references.

A developer who has written a GitHub Actions workflow can write a W3.io workflow without learning anything new. The difference is what happens after deployment. A GitHub Actions workflow runs on a centralized runner. A W3.io workflow runs on a distributed validator network with BFT consensus on every step.

The following example shows a non-trivial workflow that monitors an Ethereum wallet for large inbound transfers, screens the sender for sanctions compliance, logs the event to a database for audit, and sends a notification. It uses four ecosystem partners (Chainalysis for compliance, Space and Time for data, a Redis cache for deduplication, and an email action for alerting) alongside native Ethereum actions.

```yaml
name: Large transfer compliance monitor
on:
  schedule:
    - cron: '*/5 * * * *'
jobs:
  monitor:
    steps:
      - name: Check recent transfers
        id: transfers
        uses: w3-io/w3-sxt-action@v1
        with:
          command: query
          sql: >
            SELECT tx_hash, from_address, value
            FROM ethereum.erc20_transfers
            WHERE to_address = '${{ secrets.WATCHED_WALLET }}'
            AND value > 10000000000
            AND block_timestamp > NOW() - INTERVAL '5 MINUTES'

      - name: Deduplicate against cache
        id: dedup
        if: steps.transfers.outputs.row_count > 0
        uses: w3-io/w3-redis-action@v1
        with:
          command: sismember
```

```
          url: ${{ secrets.REDIS_URL }}
          key: processed-transfers
          value: ${{ steps.transfers.outputs.rows[0].tx_hash }}

      - name: Screen sender address
        id: compliance
        if: steps.dedup.outputs.result == '0'
        uses: w3-io/w3-chainalysis-action@v1
        with:
          address: ${{ steps.transfers.outputs.rows[0].from_address }}

      - name: Log to audit trail
        if: steps.compliance.outputs.risk != 'sanctions'
        uses: w3-io/w3-sxt-action@v1
        with:
          command: execute
          sql: >
            INSERT INTO compliance_log
            (tx_hash, sender, amount, risk_level, checked_at)
            VALUES ('${{ steps.transfers.outputs.rows[0].tx_hash }}',
                    '${{ steps.transfers.outputs.rows[0].from_address }}',
                    ${{ steps.transfers.outputs.rows[0].value }},
                    '${{ steps.compliance.outputs.risk }}',
                    NOW())

      - name: Mark as processed
        uses: w3-io/w3-redis-action@v1
        with:
          command: sadd
          url: ${{ secrets.REDIS_URL }}
          key: processed-transfers
          value: ${{ steps.transfers.outputs.rows[0].tx_hash }}

      - name: Alert on high risk
        if: steps.compliance.outputs.risk == 'high'
        uses: w3-io/w3-email-action@v1
```

```
        with:
          to: compliance@example.com
          subject: >
            High risk transfer detected:
            ${{ steps.transfers.outputs.rows[0].tx_hash }}
```

Every step in this workflow executes on a different validator, selected by consensus. The compliance check result, the database write, and the alert are all attested by the committee and included in the settlement receipt. An auditor can later prove that the compliance check happened, what it returned, and when, by verifying the receipt against the on-chain epoch root.

A workflow definition includes:

- **Name**: human-readable identifier for the workflow
- **Trigger** (`on:`): the event that initiates execution
- **Jobs**: one or more named job blocks, each containing a sequence of steps
- **Steps**: ordered operations within a job, each invoking an action or running a command
- **Environment** (`env:`): key-value pairs scoped to the workflow, job, or individual step
- **Secrets**: sensitive values fetched at runtime from a secrets provider, never stored in the workflow definition

4.2 Triggers

Every workflow begins with a trigger. W3.io supports four trigger types.

Cron schedule (`schedule`). Time-based triggers that fire on a cron expression. A workflow triggered every five minutes, every hour, or once a day. The cron expression follows standard POSIX syntax. Useful for batch processing, periodic data aggregation, and scheduled reporting.

```
on:
  schedule:
    - cron: '*/5 * * * *'
```

RPC dispatch (`workflow_dispatch`). On-demand triggers fired by an API call. A user, an application, or an AI agent sends a request to the W3.io RPC endpoint with optional input parameters. The workflow runs once with the provided inputs. Useful for user-initiated actions like deploying a contract, processing a payment, or running an analysis.

```
on:
  workflow_dispatch:
    inputs:
      token_address:
```

```
        description: 'Token to analyze'
        required: true
      chain:
        description: 'Blockchain network'
        default: 'ethereum'
```

Chain events. Blockchain event triggers that fire when a specific on-chain event occurs. An ERC-20 transfer, a contract deployment, a governance vote. The validator network monitors the specified chain for matching events and initiates the workflow when one is detected. Currently supported for Ethereum and Solana.

Oracle feeds. External data triggers based on oracle price updates or threshold crossings. When a Pyth price feed crosses a defined boundary, the workflow fires. Useful for automated trading, liquidation monitoring, and alerts.

4.3 Step Kinds

Each step in a workflow invokes one of five step kinds.

Run (`run:`). Executes a shell command inside an isolated container. The command runs in a Docker container with a defined image, environment variables, and mounted volumes. Output is captured from stdout and parsed as step outputs. This is the most flexible step kind, suitable for custom logic, data transformation, and any operation not covered by a specialized action.

```
- name: Process data
  run: |
    echo "Processing ${{ steps.query.outputs.count }} records"
    python3 transform.py --input data.json
```

Uses (`uses:`). Invokes a published action by reference. The action is identified by its namespace, name, and version. Inputs are passed via the `with:` block. Outputs are available to subsequent steps via `${{ steps.<id>.outputs.<key> }}`. This is how ecosystem partner capabilities are accessed.

```
- name: Query blockchain data
  id: query
  uses: w3-io/w3-sxt-action@v1
  with:
    command: query
    sql: SELECT * FROM ethereum.transactions LIMIT 10
```

Ethereum (`ethereum:`). Native blockchain operations on Ethereum and EVM-compatible chains. Supports balance queries, token transfers and approvals, contract deployment and calls, event queries, NFT operations, and ENS name resolution. The `network` parameter selects the target chain.

```yaml
- name: Check balance
  ethereum:
    action: get-balance
    network: ethereum
    params:
      address: "0x742d35Cc6634C0532925a3b844Bc9e7595f2bD18"
```

Solana (`solana:`). Native blockchain operations on Solana. Supports balance queries, token account lookups, program invocation, SOL and SPL token transfers, and transaction monitoring.

Bitcoin (`bitcoin:`). Native blockchain operations on Bitcoin. Supports balance queries, UTXO management, fee estimation, and transaction construction.

4.4 Expressions and Data Flow

W3.io's expression engine evaluates dynamic values at runtime using the `${{ }}` syntax. Expressions can reference:

- **Step outputs**: `${{ steps.query.outputs.result }}` accesses a named output from a previous step
- **Inputs**: `${{ inputs.token_address }}` accesses a trigger input parameter
- **Environment**: `${{ env.API_KEY }}` accesses an environment variable
- **Secrets**: `${{ secrets.PRIVATE_KEY }}` accesses a secret value (redacted in logs)
- **Job context**: `${{ job.status }}` reflects the current job state

The expression language supports functions for string manipulation (`contains`, `startsWith`, `endsWith`, `format`), data conversion (`toJSON`, `fromJSON`), and status checks (`success()`, `failure()`, `always()`).

Data flows between steps through outputs. Each step can produce named outputs that subsequent steps reference by step ID. This creates a typed data pipeline where each step receives exactly the data it needs from previous steps, without shared mutable state.

4.5 Conditionals and Failure Handling

Steps can be conditionally executed using the `if:` field.

```yaml
- name: Alert on failure
  if: failure()
  uses: w3-io/w3-email-action@v1
  with:
    to: ops@example.com
    subject: Workflow failed
```

The `continue-on-error` field controls whether a step failure stops the workflow or allows subsequent steps to execute. Combined with status check functions (`success()`, `failure()`, `always()`), this enables error handling patterns like cleanup operations, fallback logic, and notification on failure.

Each step has a configurable timeout via `timeout-minutes`. If a step exceeds its timeout, the execution backend terminates it and the step is marked as failed. The default timeout is 10 minutes. The effective timeout for any step is the minimum of the step's configured timeout and the remaining time in the job's overall timeout.

4.6 Compilation and Deployment

Workflow YAML is not interpreted directly by validators. It goes through a compilation pipeline.

Parse: the YAML is parsed into an abstract syntax tree. Structural errors (invalid keys, missing required fields, malformed expressions) are caught here.

Validate: semantic validation checks type consistency, expression references, step ID uniqueness, and action compatibility. Invalid expression references (referencing a step that doesn't exist, or an output that an action doesn't produce) are caught before deployment.

Compile: the validated AST is compiled into a canonical form. Comments, formatting, and whitespace are stripped. The result is a deterministic representation of the workflow's semantic content.

Hash: the compiled form is hashed. This `definition_hash` is the workflow's identity for settlement purposes. Two workflows with identical semantics but different formatting produce the same hash. The hash is stored on-chain in the WorkflowRegistry contract, creating an immutable link between the deployed workflow and its on-chain provenance.

Deploy: the compiled workflow is distributed to the validator network. Validators store the definition and begin monitoring for the specified trigger events.

5 Execution

5.1 The Separation of Concerns

W3.io's execution model separates **what** to execute from **how** to execute it. The workflow runtime decides what happens at each step: which action to invoke, what inputs to pass, how to parse the outputs. The execution backend decides how it happens: which container to run, how to enforce timeouts, how to handle failures.

This separation is implemented through two traits in the codebase.

The **Syscalls** trait defines five operations that the runtime can invoke: `run` (shell commands), `uses` (published actions), `ethereum`, `solana`, and `bitcoin` (native chain operations). The runtime calls these without knowing anything about the underlying execution infrastructure.

The **Backend** trait defines two operations that the syscalls layer delegates to: `prepare` (make the execution environment ready) and `execute` (run a command and return the result). The backend handles isolation, timeouts, resource management, and cleanup.

Today, W3.io ships with a Docker backend. The architecture is designed so that a Firecracker backend (microVM isolation) or a WASM backend can be added without changing the runtime or the syscalls layer.

5.2 Docker Execution

When a validator is selected to execute a workflow step, the Docker backend handles the full lifecycle.

Preparation. The backend pulls the required container image (if not already cached) and prepares the execution environment. Image pulls are deduplicated across concurrent workflows. If two workflows need the same image at the same time, only one pull happens.

Container naming. Each container gets a deterministic name based on the workflow's trigger hash, job ID, step index, and a timestamp. This makes containers identifiable in Docker logs, and it means the backend can kill a specific container by name if the process becomes unresponsive.

Execution. The step's command runs inside the container with the workflow's environment variables, secrets (injected via a temporary env file that is deleted after startup), and any required filesystem mounts. Standard output and standard error are captured as the step's log output.

Timeout enforcement. Every step has a timeout (configurable via `timeout-minutes:`, default 10 minutes). The backend uses `tokio::select!` with fair polling to race the step execution against a deadline. If the deadline fires first, the backend initiates a shutdown sequence:

1. Send SIGTERM to the Docker process (Docker forwards this to the container's PID 1)

2. Drain stdout and stderr during a 10-second grace period (prevents pipe deadlock where a process writing during shutdown blocks on full pipes)
3. If the process hasn't exited after the grace period, send SIGKILL
4. Unconditionally call `docker kill` by container name as a final defense

This escalation ensures that a hung container never blocks consensus indefinitely. The step is marked as failed with a timeout error, and the workflow's failure handling logic (continue-on-error, if conditions) determines what happens next.

Output parsing. After execution completes, the backend parses the container's output. Step outputs (key-value pairs written to a designated output file inside the container) are extracted and made available to subsequent steps via the expression engine. Environment modifications (variables written to the environment file) are merged into the workflow's environment for downstream steps.

5.3 Step Identity

Every execution carries typed identity fields that flow from the consensus layer through the entire call chain: the trigger hash (which workflow run), the job ID (which job within the run), and the step index (which step within the job). These fields appear in every backend log line, making it possible to trace a specific step's execution across container logs, P2P messages, and consensus records using the same identifiers.

This is not just a debugging convenience. The step identity is part of the settlement receipt. The trigger hash, combined with the step index, uniquely identifies a specific step execution within the protocol's history. When a receipt is verified against an on-chain epoch root, the step identity ties the cryptographic proof to a specific observable event.

5.4 Future Backends

The Backend trait is designed to support execution environments beyond Docker.

Firecracker. Firecracker microVMs provide stronger isolation than containers. Each step would execute in its own lightweight VM, booted in milliseconds, with a dedicated kernel. The `prepare` method would handle VM boot and filesystem setup. The `execute` method would run the command inside the VM. This is the planned path for production workloads where stronger isolation is required.

WASM. WebAssembly runtimes offer sandboxed execution without the overhead of containers or VMs. A WASM backend would compile actions to WASM modules and execute them in a sandboxed runtime. This is a longer-term consideration for lightweight actions where container startup overhead is disproportionate to the computation.

Both future backends would implement the same `prepare` and `execute` interface. The runtime, the consensus layer, and the settlement layer would not change. Only the isolation boundary moves.

6 Consensus

6.1 BOSCO

W3.io uses a single Byzantine fault-tolerant consensus algorithm called BOSCO (Song and Renesse 2008) for all protocol-level agreement. BOSCO builds on the foundational work of practical BFT (Castro and Liskov 1999) and tolerates up to f faulty or adversarial validators in a committee of 3f+1 members. In a 10-member committee, up to 3 members can be Byzantine (offline, malicious, or sending conflicting messages) and the protocol still reaches correct consensus.

BOSCO operates in three phases.

Propose. The committee leader constructs a proposal and broadcasts it to all committee members. The proposal contains the value being decided on (which validator should run a step, what the attestation set looks like, which receipts to include in an epoch) along with the leader's identity and a round number. The leader is selected deterministically from the committee using the round number, so all honest members agree on who the current leader is.

Vote. Each committee member receives the proposal, validates it against their local state (does the proposed runner exist in the validator set? does the receipt hash match what they received via gossipsub?), and broadcasts a vote. A vote either confirms the proposal or rejects it. Votes are signed and include the round number to prevent cross-round replay. Each member collects votes from other members as they arrive over P2P.

Decide. When a member has collected 2f+1 matching votes for the same proposal in the same round, the decision is final. The member applies the decision locally (advance the state machine, record the attestation, close the epoch). The 2f+1 threshold guarantees that any two quorums overlap by at least one honest member, which means two conflicting decisions in the same round are impossible.

View change. If the leader fails to propose within a configurable timeout, any member can initiate a view change. The round number increments, a new leader is selected deterministically, and the protocol restarts from the Propose phase with the new leader. This is automatic. No operator intervention is needed. A faulty leader costs one timeout period (typically seconds), not a stalled workflow.

View changes are critical for liveness. Without them, a single offline or malicious leader could block all workflow execution indefinitely. With them, the worst case is a brief delay while leadership rotates. The protocol cycles through leaders until it finds one that responds.

BOSCO is not a novel construction. It follows the established BFT pattern found in production systems across the blockchain industry (Castro and Liskov 1999). W3.io's implementation is 2,720 lines of Rust (Matsakis and Klock 2014), battle-tested under multi-node deployments with injected Byzantine faults including equivocating leaders, message delays, and partitioned networks.

W3.io chose a single consensus algorithm rather than offering pluggable consensus per workflow. This is deliberate. Pluggable consensus adds complexity without proportional benefit. The security properties of the network depend on one well-tested algorithm, not a menu of options with varying guarantees. BOSCO is used everywhere consensus is needed. The contexts differ, but the mechanism is the same.

6.2 The Workflow State Machine

When a workflow is triggered, W3.io's consensus engine drives it through a four-state machine. Each state represents a phase of the workflow step lifecycle, and each transition requires committee agreement.

State 1: Awaiting Trigger. A trigger event arrives: a cron tick, an RPC call, a chain event, or an oracle update. The protocol must first confirm that the trigger is legitimate. For RPC triggers, this means verifying the caller's signature against the namespace's authorization policy. For chain events, this means confirming the event actually occurred on-chain through a monitor group (a small committee of validators assigned to watch that specific event type).

Once the trigger is confirmed, a workflow committee is selected from the active validator set using the deterministic shuffle described below. The committee members receive the trigger evidence and the workflow definition. The run begins.

State 2: Awaiting Step Runner Proposals. For each step in the workflow, the committee must agree on which validator executes it. The committee leader examines the eligible validators (those online, with sufficient capability, and not already overloaded) and proposes a runner. Committee members verify that the proposed runner is eligible and vote to confirm.

If the leader fails to propose within the round timeout, view change rotates leadership. If the selected runner accepts but then fails to return a result within the step timeout, the committee marks the runner as failed and the leader proposes a new runner. After M consecutive runner failures for the same step (configurable, default 3), the workflow itself is marked as failed. This prevents infinite retry loops on steps that are fundamentally broken.

State 3: Awaiting Step Execution. The selected runner executes the step using the execution backend (Section 4). This is the only phase where actual computation happens. The runner pulls the container image (if needed), injects the step's inputs and environment, runs the command, and captures the outputs.

When execution completes, the runner broadcasts the result to the committee: the step's outputs, the execution duration, the exit code, and a hash of the produced workflow block. The workflow block contains the step's inputs, outputs, the runner's identity, and the previous block's hash, forming a per-run hashchain.

State 4: Awaiting Step Attestation. Committee members independently assess the runner's result. Each member produces an attestation with one of three values:

- **Confirmed**: the result appears correct based on the member's local validation
- **Rejected**: the result is inconsistent with what the member expected (for example, the output hash doesn't match a deterministic recomputation)
- **Indeterminate**: the member cannot determine correctness (for example, the step interacted with an external API that the member cannot independently verify)

The committee runs BOSCO to agree on the canonical attestation set. A deterministic decision rule produces the final verdict from the agreed assessments. If the majority confirmed, the step passes and the state machine advances to State 2 for the next step. If the majority rejected, the step fails and the workflow's failure handling logic determines whether execution continues.

When all steps complete, the workflow produces a receipt containing the trigger hash, the final state (completed or failed), timing data, the committee seed, the definition hash, the namespace, and the hashes of all consensus blocks. This receipt enters the settlement pipeline described in Section 6.

6.3 Committee Selection

Committee selection determines which validators participate in consensus for a given workflow execution. The selection must be deterministic (all honest nodes compute the same committee), unpredictable (no party can influence or predict committee composition before the trigger), and capability-aware (only validators with the required capabilities are eligible).

W3.io achieves this with a seeded Fisher-Yates shuffle.

The seed is derived from two values: the cumulative settlement root and the trigger hash. The cumulative root is the global sparse merkle tree root from the most recently settled epoch. It depends on every workflow receipt ever settled across all namespaces and all prior epochs. An attacker trying to predict or influence committee membership would need to control the settlement of all workflows across the entire network, not just the target workflow. The trigger hash adds per-run entropy that is unique to this specific workflow execution. Together they function like a verifiable random function output: deterministic, unpredictable before the epoch settles, and publicly verifiable by anyone who knows the on-chain root.

The seed feeds a ChaCha20 pseudorandom number generator (Nir and Langley 2018), which drives the Fisher-Yates shuffle over the eligible validator set. ChaCha20 is a stream cipher used here as a cryptographically secure PRNG. It replaced an earlier linear congruential generator that was predictable and trivially invertible. The security of committee selection depends on the PRNG being non-invertible. ChaCha20 provides this.

Before shuffling, the validator set is filtered by capabilities. Each validator declares a capability bitmap at registration: whether it maintains an Ethereum WebSocket connection, an Avalanche WebSocket connection, GPU compute access, or other resources. A workflow that triggers on Ethereum events requires validators with the Ethereum WebSocket capability. The shuffle operates only on validators whose capabilities satisfy the workflow's requirements, ensuring that selected committee members can actually perform the work.

The minimum committee size in production is configurable through governance. Larger committees provide stronger Byzantine fault tolerance (more members must be compromised) but increase the P2P message overhead per step. The default minimum is 10, giving f=3 tolerance.

6.4 Four Consensus Contexts

BOSCO is used in four distinct contexts within the protocol. The algorithm is identical in each case. What differs is the proposal value, the committee composition, and the committee lifetime.

Step runner selection. For each workflow step, the per-run committee agrees on which validator executes it. The proposal value is the selected runner's identity. This is the highest-stakes use of BOSCO because it directly determines who controls the step's execution. Committee size follows the governance-defined minimum (default 10 members, f=3). The committee is formed at trigger time and persists for the duration of the workflow run.

Step attestation. After execution, the same per-run committee agrees on the canonical assessment of the result. The proposal value is the attestation set (each member's confirmed/rejected/indeterminate verdict). This separates execution from evaluation: the runner produces the result, but the committee decides whether to accept it. A single compromised runner cannot force a bad result through consensus because the committee independently evaluates the output.

Trigger confirmation. Dedicated monitor groups run BOSCO to confirm external events before dispatching workflow execution. A monitor group is a small committee (7 members, f=2) assigned to a specific trigger type. Multiple workflows that watch the same event (for example, the same ERC-20 Transfer event on the same contract) share a single monitor group. This prevents trigger spoofing: a single validator claiming an event occurred is not sufficient to start a workflow. The monitor group must reach BFT agreement on the trigger evidence, including temporal validation

that rejects future-dated events.

Epoch manifest agreement. The aggregation committee uses BOSCO to agree on which workflow receipts are included in each settlement epoch. The aggregation committee is larger (30 members, f=9) and longer-lived (persists for an entire epoch term, not just one workflow run). The proposal value is the receipt manifest: a sorted list of receipt hashes. Once the manifest is agreed upon, every honest member independently builds the same sparse merkle tree from the same receipts, producing the same cumulative root. This root is then signed and submitted to the L1 contract (Section 6).

In every context, the same properties hold: 2f+1 agreement is required, leader failure triggers automatic view change, and the decision is deterministic given the same inputs. W3.io does not need four consensus algorithms. It needs one good one, applied consistently.

7 Settlement

7.1 Why Settlement Matters

W3.io workflows execute off-chain. Validators run containers, reach consensus on results, and produce outputs. But without settlement, verifiability ends at the validator set. A third party who was not part of the consensus has no way to independently confirm that a workflow ran, what it produced, or when it executed.

Settlement bridges this gap. It takes the outputs of off-chain consensus and anchors them to a public blockchain, creating a permanent, independently verifiable record. After settlement, anyone with access to the L1 can verify a workflow execution. No access to the validator network required. No trust in any specific validator.

This is the difference between "the validators say it happened" and "the chain proves it happened."

7.2 Receipts

When a workflow run completes, the consensus engine produces a **receipt**. The receipt is a compact summary of everything that matters about the execution. It contains 15 fields:

- **version**: receipt format version (currently 1)
- **namespace**: the namespace that owns the workflow
- **workflow_name_hash**: keccak256 hash of the workflow name
- **trigger_hash**: unique identifier for this specific run
- **definition_hash**: hash of the compiled workflow definition, matching the on-chain registry
- **l1_anchor_hash**: the L1 block hash at the time the workflow was triggered
- **final_block_hash**: hash of the last consensus block in the run's hashchain

- **end_state**: completed or failed
- **failure_reason**: if failed, why (runner timeout, attestation rejection, step error)
- **failure_data_hash**: hash of the failure details for diagnostic retrieval
- **actions_used**: sorted list of namespace-scoped action ID hashes used in the run
- **step_count**: total number of steps executed
- **start_time**: when the workflow started (UTC milliseconds)
- **end_time**: when it completed
- **committee_seed**: the seed used for committee selection, enabling anyone to reconstruct which validators participated

The receipt is ABI-encoded using the same layout as Solidity's `abi.encode`, producing a byte-for-byte match between the Rust and Solidity implementations. The receipt hash is `keccak256(abi.encode(receipt))`. This hash is what gets inserted into the sparse merkle tree and ultimately anchored on-chain.

The ABI encoding compatibility is verified by a golden test: the Rust implementation's output is compared against the output of `cast abi-encode` (the Solidity reference encoder) for identical inputs. If the encoding ever drifts, the test fails before the code ships.

7.3 Epochs

Receipts are not settled individually. They are batched into **epochs**. An epoch is a fixed-duration window during which receipts accumulate. When the epoch closes, the aggregation committee agrees on the receipt manifest (the list of receipt hashes to include), builds the merkle tree, and submits the commitment to the L1.

Epoch mechanics:

- Epochs are numbered sequentially. Empty epochs (no workflows ran) are skipped. The contract accepts the next non-empty epoch regardless of how many empty epochs elapsed.
- Each epoch references the previous epoch's cumulative root, creating a hashchain of epochs on the L1. This prevents replay, reordering, and forking.
- A receipt that arrives after the epoch closes goes into the next epoch. The cutoff is hard. There is no grace period. Late receipts are not lost, they are deferred.
- The receipt manifest is agreed upon by the aggregation committee using BOSCO consensus (Section 5). This ensures all honest validators build the tree from the same receipt set.

Batching into epochs serves two purposes. First, it amortizes the gas cost of L1 submission across many workflow executions. At 100 workflows per epoch, the per-workflow settlement cost approaches zero. Second, it produces a cumulative root that covers the full history of all settled workflows, not just the current epoch. This is what makes historical verification possible without

scanning the entire chain.

7.4 The Two-Level Sparse Merkle Tree

W3.io uses a two-level cumulative sparse merkle tree (Gao et al. 2021) to organize settled receipts.

Level 1: Global tree. The leaves are namespace roots, keyed by `keccak256("ns" || namespace_id)`. Each leaf's value is the root of that namespace's own tree. The global tree is sparse: only namespaces with at least one settled receipt have leaves. A namespace created on-chain but never used does not appear in the tree.

Level 2: Per-namespace trees. Each namespace has its own tree. The leaves are receipt hashes, keyed by `keccak256(version || namespace_id || trigger_hash || workflow_name_hash)`. Each leaf's value is the receipt hash.

Both trees use keccak256 hashing for EVM compatibility. The tree implementation wraps the Jellyfish Merkle Tree library (originally developed for the Diem/Aptos blockchain), which provides efficient path compression, incremental updates, and both inclusion and non-inclusion proofs.

The cumulative root after processing epoch N is the on-chain root for epoch N. It reflects the entire history of all receipts across all namespaces, not just epoch N's receipts. Like Ethereum's state trie, each epoch's root is a snapshot of the full world state at that point.

This structure enables three types of verification:

Inclusion proof. Prove that a specific workflow ran and settled. The verifier needs the receipt, a per-namespace merkle proof (receipt exists in the namespace tree), and a global merkle proof (namespace root exists in the global tree). Two proofs, checked against the on-chain root. Typical proof size is 2-3 KB.

Non-inclusion proof. Prove that a workflow did NOT run. The sparse merkle tree supports non-membership proofs: given a key, prove that no leaf exists at that position. This is critical for dispute resolution and billing ("you claim we didn't run your workflow; here's cryptographic proof that we did" or "you claim you ran it; prove it against the root").

Composability. Other smart contracts can verify workflow results on-chain. The `SMTVerifier` contract exposes a `verifyWorkflowResult()` function that accepts a receipt, two proofs, and an epoch number, and returns whether the receipt is valid against the on-chain root. A DeFi protocol can gate actions on workflow completion: "release payment only if this compliance workflow settled successfully."

7.5 L1 Anchoring

The final step in settlement is submitting the epoch commitment to the L1 contract. This is a two-phase process.

Phase 1: Manifest agreement. The aggregation committee leader proposes the receipt manifest for the epoch. Committee members run BOSCO to reach consensus on which receipts are included. Once agreed, every honest member independently builds the same sparse merkle tree from the same receipts (deterministic: same receipts in the same order produce the same root).

Phase 2: Root signing. After building the tree, committee members produce a quorum signature over the epoch tuple: chain ID, contract address, epoch number, validator set version, cumulative root, previous root, receipt count, and namespace count. The chain ID and contract address serve as a domain separator, preventing signatures from one deployment being replayed on another. The quorum signature is an aggregate BLS12-381 signature (Boneh et al. 2022) verified on-chain via EIP-2537 precompiles (Vlasov and Olson 2020).

The `EpochSettlement` contract verifies:

- The epoch number is greater than the last submitted epoch (monotonically increasing)
- The previous root matches the stored root from the last epoch (chaining, no forks)
- The quorum signature is valid against the known validator set
- The validator set version matches the current on-chain version

If all checks pass, the contract stores the new cumulative root, emits an `EpochSubmitted` event, and the epoch is settled. The cumulative root is now publicly queryable by any contract or off-chain verifier.

Submission incentives. The manifest BOSCO leader has priority to submit during a configurable window (default 30 seconds). After the window expires, any validator can submit, incentivized by a gas reimbursement bounty. This ensures that a single faulty leader cannot block settlement. The contract's idempotent design means racing submitters are harmless: the first valid submission is accepted, duplicates revert cheaply.

7.6 Crash Recovery

Settlement is designed to survive node failures without losing data.

W3.io uses a write-ahead log (WAL) to track receipts and epoch state. Every receipt written to the WAL is durable (when backed by MDBX persistent storage). Every epoch status transition (open, closed, submitted, settled, failed) is recorded. If a node crashes mid-epoch, it restarts, opens the WAL (which self-heals to the last committed transaction), checks the L1 for the latest settled epoch, and resumes from where it left off.

The recovery sequence:

1. Open the WAL. MDBX guarantees consistency after unclean shutdown.
2. Read the latest settled epoch from the L1 contract. This is the source of truth.
3. Check the WAL for unsettled epochs. Any epoch in Closed or Submitted status needs re-processing or re-submission.
4. Backfill missing receipts from peers for the last 2 epochs.
5. Re-process if needed. Same receipts plus same manifest produces the same root. Deterministic replay is safe.
6. Check for in-flight L1 submissions. If an epoch was submitted but the WAL doesn't know, scan for it on-chain. The contract rejects duplicates.

The worst case after a power failure is a brief delay while the node catches up. No data is lost. No re-sync from genesis is required.

8 Cryptography

8.1 Building on Giants

W3.io does not invent new cryptographic primitives. Every algorithm used in the protocol is a well-established, audited, production-grade construction with years of deployment in other systems. The protocol's security properties are inherited from these foundations, not asserted by novel constructions.

This is a deliberate choice. Novel cryptography is interesting research but a poor foundation for a production protocol. The attack surface of a new primitive is unknown by definition. The attack surface of BLS12-381 is understood by thousands of researchers and audited by every major Ethereum consensus client team. W3.io benefits from that scrutiny without paying for it.

The following table summarizes the cryptographic primitives used in the protocol, their purpose, and their provenance.

Primitive	Purpose	Standard/Library	Used By
BLS12-381	Quorum signatures on epoch submissions, receipt signing, trigger confirmation, validator proof of possession	blst (Supranational 2023), draft-irtf-cfrg-bls-signature (Boneh et al. 2022)	Ethereum 2.0 (Lighthouse, Prysm, Teku, Lodestar, Nimbus), Cosmos, Polkadot
keccak256	Receipt hashing, namespace ID derivation, SMT key derivation, ABI encoding	FIPS 202 (NIST 2015)	Ethereum (native hash function), Solidity `abi.encode`
ChaCha20	Deterministic committee selection PRNG	RFC 8439 (Nir and Langley 2018)	TLS 1.3, WireGuard, Linux kernel CSPRNG
Ed25519	Validator identity keys, P2P message signing	RFC 8032	libp2p, Signal, SSH, Tor
SHA-256	Workflow run block hashchaining, trigger hash derivation	FIPS 180-4	Bitcoin, TLS, nearly every production system
Jellyfish Merkle Tree	Two-level cumulative sparse merkle tree for settlement state	Diem/Aptos (Gao et al. 2021)	Aptos, Penumbra
SSZ	Wire format for P2P message serialization	Ethereum SSZ specification (Foundation 2019)	Ethereum 2.0 consensus layer

8.2 BLS12-381

W3.io uses BLS12-381 signatures for all protocol-level aggregate signing. BLS (Boneh-Lynn-Shacham) signatures have a unique property that makes them ideal for quorum-based systems: multiple individual signatures can be aggregated into a single signature of constant size, and the aggregate can be verified against the aggregate public key in a single operation. A quorum of 20 validators produces a signature no larger than one validator's signature.

W3.io's BLS implementation wraps the `blst` library (Supranational 2023), the same library used by every major Ethereum 2.0 consensus client. The protocol uses the min-pk scheme (public keys in G1, signatures in G2), matching Ethereum 2.0's choice. On-chain verification uses EIP-2537 precompiles (Vlasov and Olson 2020) for gas-efficient pairing checks.

Four domain separation tags (DSTs) prevent cross-context signature replay:

Context	DST	Purpose
Epoch quorum	`W3IO-BLS-EPOCH-V1`	Validators sign the epoch tuple for L1 submission
Receipt gossip	`W3IO-BLS-RECEIPT-V1`	Committee members sign individual receipts
Trigger confirmation	`W3IO-BLS-TRIGGER-V1`	Monitor group members sign trigger evidence
Proof of possession	`W3IO-BLS-POP-V1`	Validators sign their own public key at registration

A signature produced under one DST cannot be accepted under another, even if the underlying message bytes are identical. This is enforced at the library level: the DST is a required parameter to every sign and verify call.

Every validator must submit a proof of possession (PoP) at registration. The validator signs their own compressed G1 public key using the `W3IO-BLS-POP-V1` DST. The on-chain `ValidatorSet` contract verifies this signature before accepting the key. This prevents rogue public key attacks, where an attacker crafts a public key that cancels out honest validators' keys during aggregation.

W3.io enforces PoP at the type level. The `PopVerifiedKey` type can only be constructed by passing PoP verification. The `verify_aggregate` function requires `PopVerifiedKey` inputs, not raw public keys. This makes it a compile-time error to use an unverified key in aggregate signature verification.

8.3 The Hash Boundary

W3.io uses two hash functions for different layers of the protocol, with a clean boundary between them.

keccak256 is used for everything that touches the settlement layer or needs EVM compatibility: receipt hashing (`keccak256(abi.encode(receipt))`), namespace ID derivation (`keccak256(creator, nonce)`), SMT key derivation, and any value that will be verified by a Solidity contract. Keccak256 is Ethereum's native hash function. Using it for settlement means W3.io's proofs are directly verifiable by EVM contracts without hash function translation.

SHA-256 is used for protocol internals that never cross the EVM boundary: workflow run block hashchaining, trigger hash derivation, and monitor group seed computation. SHA-256 is used in 17 files across 8 crates in the protocol codebase.

The boundary is enforced through the type system. Settlement-layer types use `[u8; 32]` values produced by keccak256. Protocol-internal types use separate newtypes. A keccak256 hash cannot be accidentally passed where a SHA-256 hash is expected.

8.4 ChaCha20 for Committee Selection

Committee selection requires a PRNG that is deterministic (all nodes compute the same shuffle), cryptographically secure (the output cannot be predicted or inverted), and efficient (committee selection happens frequently). ChaCha20 (Nir and Langley 2018) meets all three requirements.

The seed is a SHA-256 hash of the cumulative settlement root concatenated with the trigger hash. This seed initializes a ChaCha20 stream cipher, which produces the random bytes needed to drive a Fisher-Yates shuffle over the validator set. The same seed always produces the same committee. Different seeds produce unpredictable committees.

ChaCha20 replaced an earlier linear congruential generator (LCG) that was trivially invertible. An attacker who observed a committee could invert the LCG to recover the seed and predict future committees. ChaCha20 does not have this property. The committee selection change is consensus-breaking: all validators must upgrade simultaneously.

References

Boneh, D., S. Gorbunov, R. Wahby, H. Wee, C. Wood, and Z. Zhang. 2022. *BLS Signatures*. Draft-irtf-cfrg-bls-signature-05. https://datatracker.ietf.org/doc/draft-irtf-cfrg-bls-signature/.

Buterin, Vitalik. 2014. *A Next-Generation Smart Contract and Decentralized Application Platform*. https://ethereum.org/en/whitepaper/.

Castro, Miguel, and Barbara Liskov. 1999. "Practical Byzantine Fault Tolerance." *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*. https://pmg.csail.mit.edu/papers/osdi99.pdf.

Foundation, Ethereum. 2019. *Simple Serialize (SSZ)*. https://ethereum.org/en/developers/docs/data-structures-and-encoding/ssz/.

Gao, Zhenhuan, Yuxuan Hu, and Qinfan Wu. 2021. "Jellyfish Merkle Tree." *Diem (Aptos) Technical Documentation*. https://developers.diem.com/docs/technical-papers/jellyfish-merkle-tree-paper/.

Matsakis, Niko, and Felix Klock. 2014. "The Rust Language." *ACM SIGAda Ada Letters* 34 (3): 103–4. https://dl.acm.org/doi/10.1145/2663171.2663188.

Nakamoto, Satoshi. 2008. *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf.

Nir, Y., and A. Langley. 2018. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439. https://www.rfc-editor.org/rfc/rfc8439.

NIST. 2015. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. FIPS 202. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

Song, Yee Jiun, and Robbert van Renesse. 2008. *Bosco: One-Step Byzantine Asynchronous Consensus*. Cornell University. https://www.cs.cornell.edu/projects/Quicksilver/public_pdfs/52180438.pdf.

Supranational. 2023. *Blst: Multilingual BLS12-381 Signature Library*. https://github.com/supranational/blst.

Vlasov, Alex, and Kelly Olson. 2020. *EIP-2537: Precompile for BLS12-381 Curve Operations*. Ethereum Improvement Proposals. https://eips.ethereum.org/EIPS/eip-2537.

Yakovenko, Anatoly. 2018. *Solana: A New Architecture for a High Performance Blockchain*. https://solana.com/solana-whitepaper.pdf.